

codysey-b4-2 / System Reliability Engineering

리눅스 시스템 장애 트러블슈팅 마스터 가이드

OS 원리부터 CLI 관제, 그리고 Post-Mortem 리포팅까지
(STUDY.md 핵심 해설서)

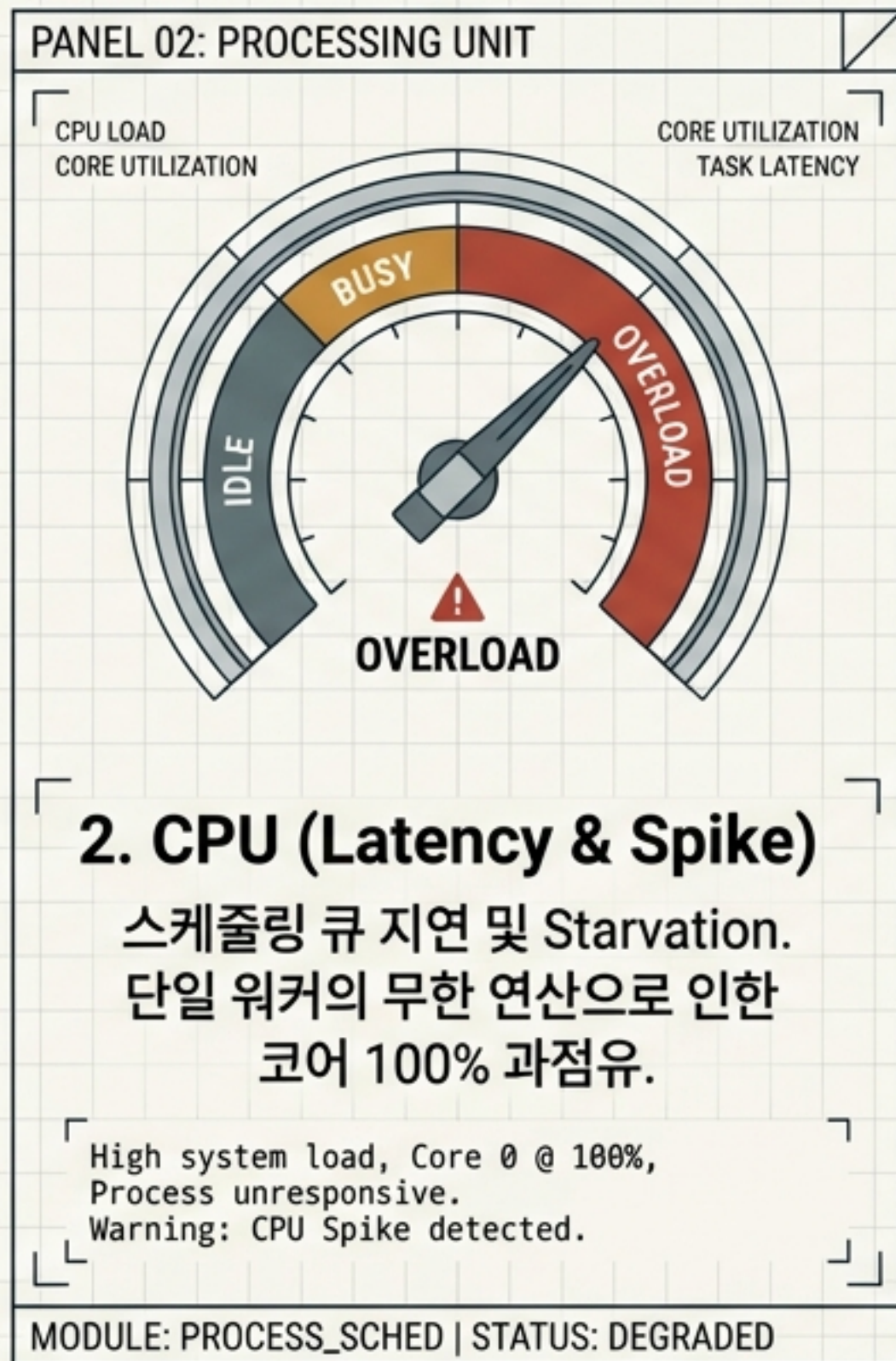
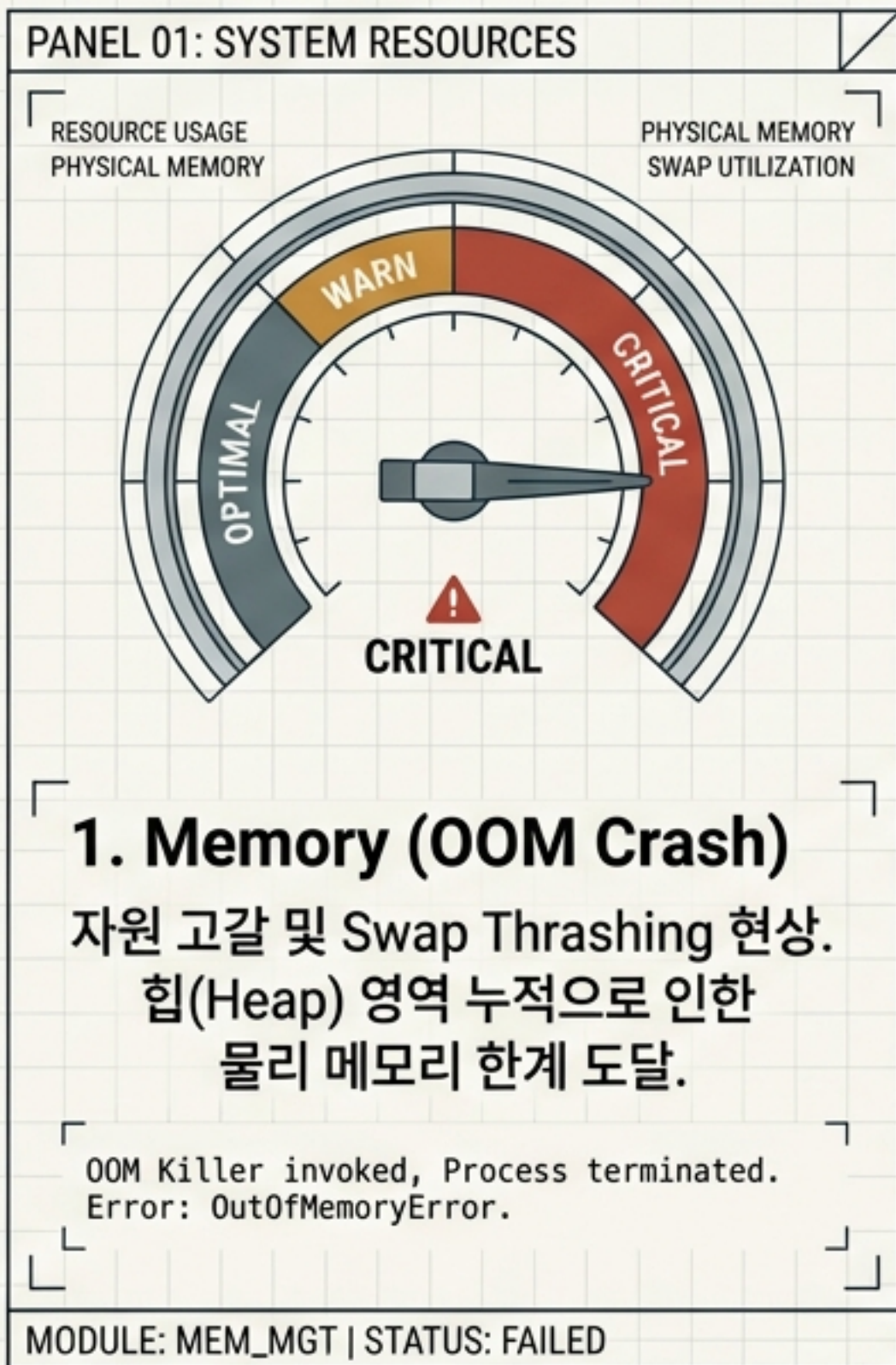
Server n

Memory Map

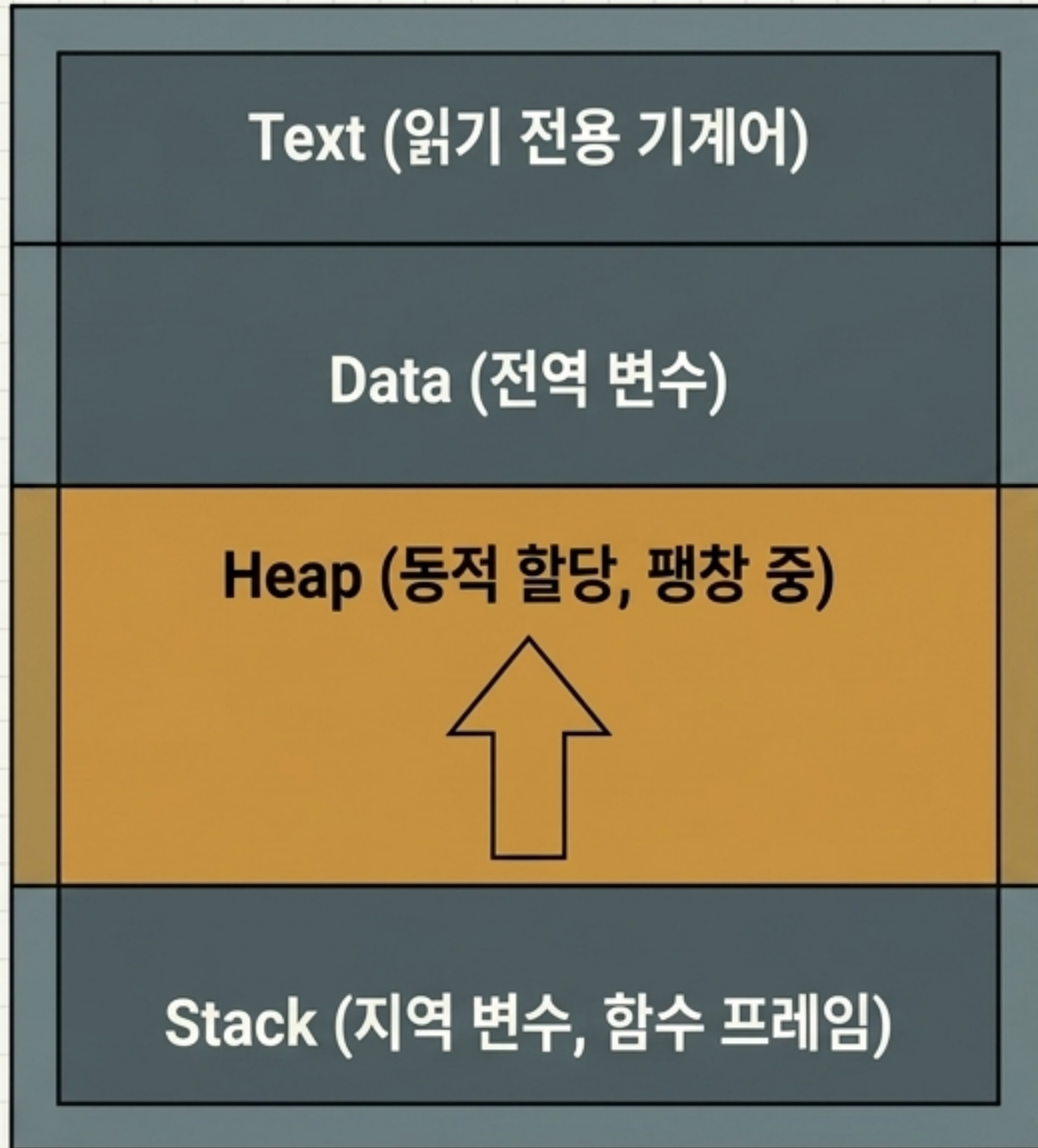
Memory

Nap

시스템을 멈추게 하는 3가지 치명적 병목



가상 메모리 공간과 진성 메모리(RSS)의 추적

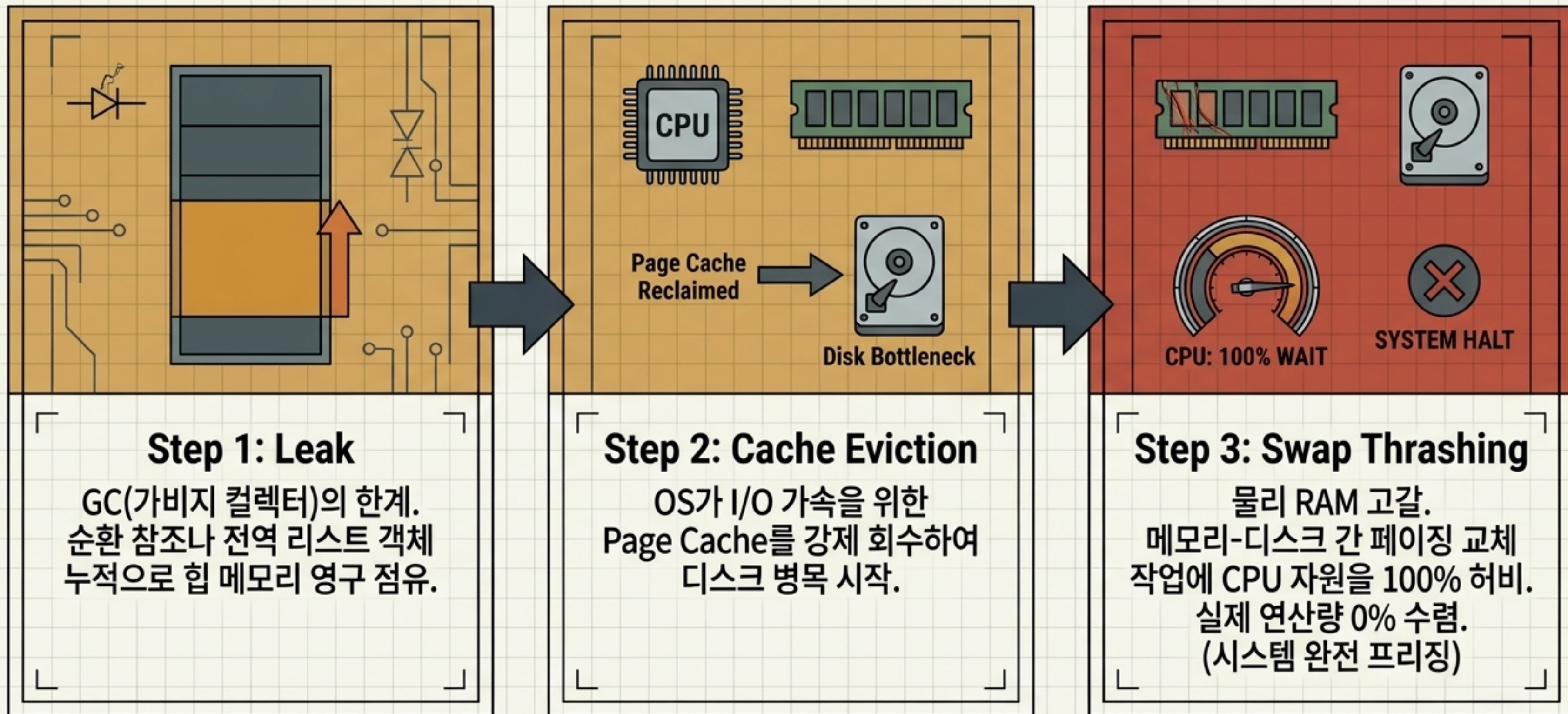


VSZ	예약된 총 가상 메모리: 미사용 페이지, 공유 라이브러리, 스왑 아웃 영역이 포함된 허수.
RSS	진성 물리 메모리: 실제 DRAM에 적재된 순수 상주 메모리. (관제의 핵심 표적)

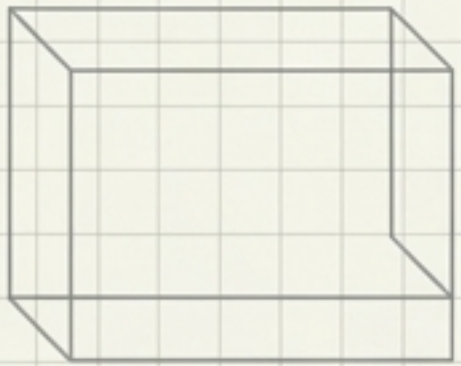
```
ps -p PID -o rss=
```

순수 물리 메모리 정밀 추출 명령어

나비효과: 메모리 누수(Leak)부터 쓰레싱(Thrashing)까지



커널의 심판관: OOM-Killer vs MemoryGuard



Linux OOM-Killer

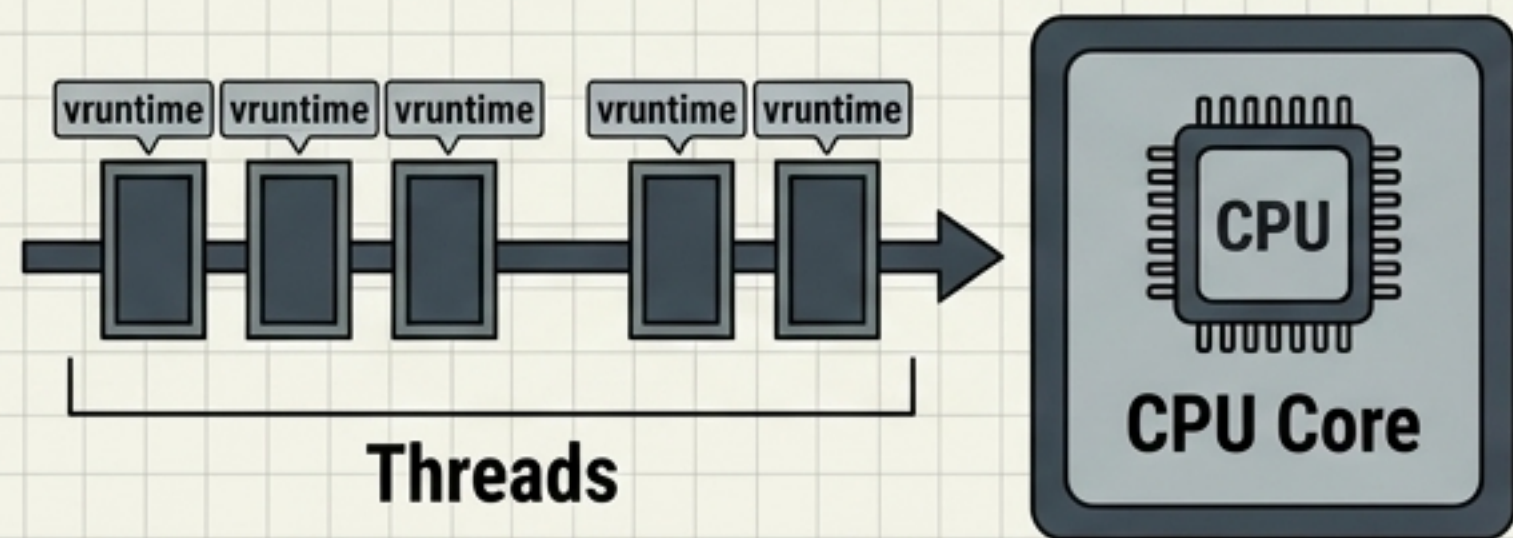
- `oom_score` 기반의 무차별 커널 도끼.
- 시스템 패닉 방지를 위해 프로세스 강제 종료. (단점: Nginx, DB 등 무고한 데몬 데몬 희생 가능성)



MemoryGuard (Self-Termination)

- 애플리케이션 자체 보호 방패.
- MEMORY_LIMIT 임계치 도달 시, 호스트 파괴를 막기 위해 스스로 SIGKILL을 호출하여 희생.

CFS 스케줄러와 실행 큐(Runqueue)의 붕괴



CFS (Completely Fair Scheduler)

vruntime(가상 실행 시간)이 가장 낮은 스레드를 최우선 배치하여 공정하게 실행.

Load Average

실행 큐(R)와 I/O 대기(D) 상태인 프로세스의 코어 수보다 높으면 **심각 대기 지연(Latency) 폭발**.

Context Switching Penalty

과점유 시 잦은 문맥 교환 발생. 레지스터 백업, TLB 플러시, 캐시 무효화로 인한 **연산력 증발**.

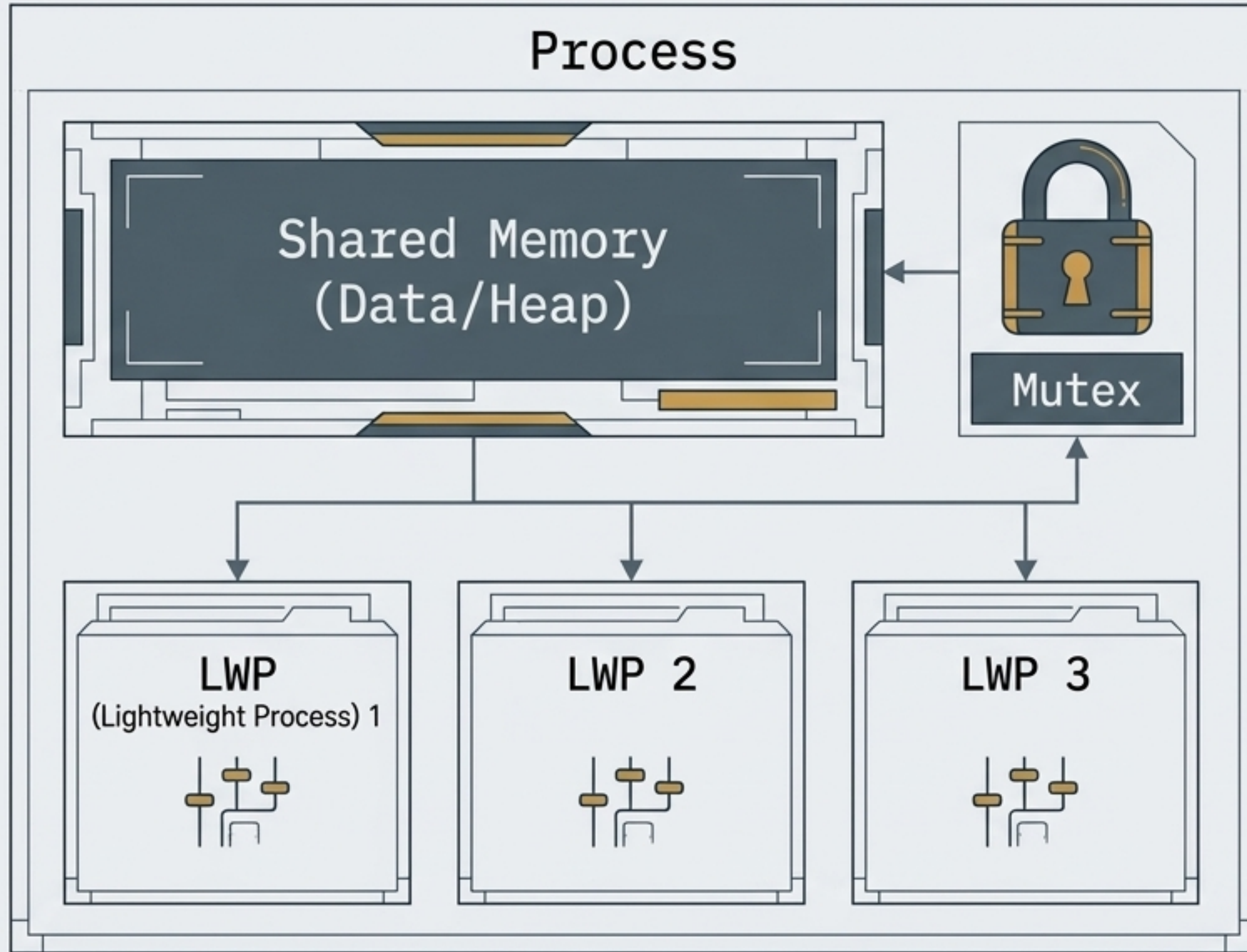
스케줄링 알고리즘 역추론 및 Watchdog 비상 중단

FCFS	Priority	Round-Robin (RR)
비선점형 (도착 순서대로 끝까지 독점)	우선순위 기반 (특정 작업 독점)	선점형. 100ms 타임 슬라이스 단위로 교차 점유. 타 스레드 작업 로그 교차 패턴으로 확정.
 배제됨	 배제됨	 확정

Watchdog 비상 중단 (Emergency Abort)

CPU 과점유 100% 지속 시, SoftIRQ(네트워크 패킷) 드롭 및 기아(Starvation) 현상을 막기 위해 감시자가 SIGTERM 비상 중단 발행.

스레드(LWP)와 상호 배제(Mutex)의 역할



LWP (Lightweight Process)

리눅스 커널은 스레드를 부모의 메모리 주소 공간을 공유하는 경량 프로세스로 취급. 고유 TID 부여.

임계 구역 (Critical Section)

데이터 무결성 붕괴를 막기 위한 보호 구역.

동기화 객체

오직 1개의 스레드만 진입 허용하는 Mutex, N개의 진입을 허용하는 Semaphore.

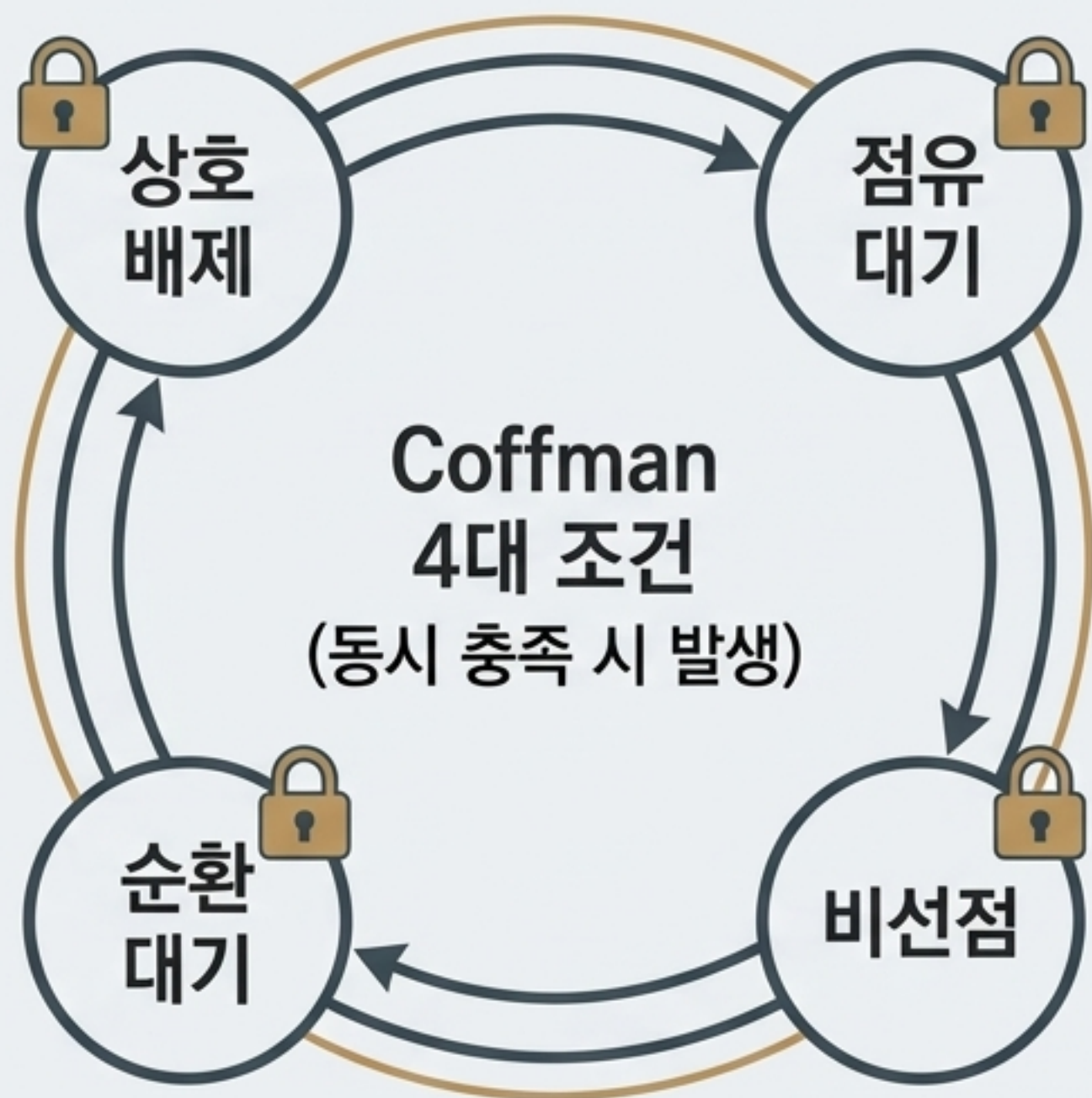


Single entry
Mutex



Multiple entry
Semaphore

교착상태(Deadlock)의 4대 완성 조건과 Futex의 늪



Futex & Kernel Sleep

유저 공간에서 락 획득 실패 시,
커널 대기 큐인 `futex_wait_queue_me`로
진입하여 영구 슬립(Sleep).

CPU 0.0%의 끔찍한 무응답 고착.

데드락 예방 기법

Lock Ordering (순서 정규화),
Timeout 기반 TryLock 도입.

프로세스 상태(STAT)와 커널 시그널 해독기

STAT 코드 분석

R : Running / Runnable

R : Running / /Runnable

S : Interruptible Sleep

S : Interruptible Sleep

D : Uninterruptible Sleep (kill -9 무시) 

D : Uninterruptible Sleep 

Z : Zombie 

Z : Zombie

종료 코드 공식: ? = 128 + Signal

SIGTERM (15) -> Exit 143

Watchdog에 의한 정상 중단 요청
(핸들러 Catch 및 정리 가능). 

SIGKILL (9) -> Exit 137

OOM-Killer / MemoryGuard에 의한
즉각 강제 살해
(Catch 절대 불가, 커널 직접 집행). 

시스템 관제 및 진단 CLI 아스널(Arsenal)

[생존 & 네트워크]

```
pgrep -f (PID 존재 확인)  
ss -tulnp (포트 바인딩 감시)
```

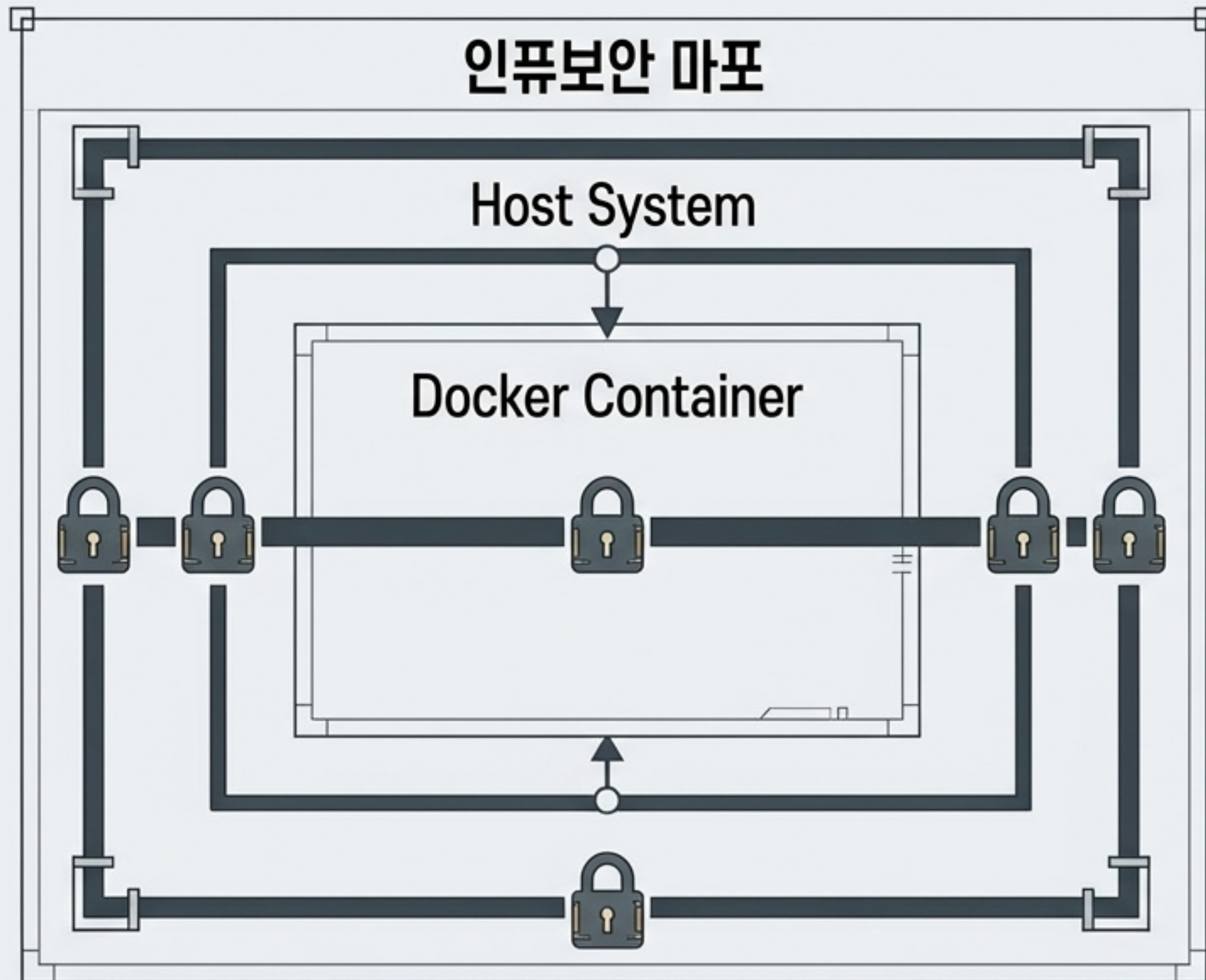
[리소스 관제 파싱]

```
ps -p PID -o rss=, %cpu=  
(순수 수치 추출)  
  
top -b -n 1  
(배치 모드 덤프)
```

[Deadlock 정밀 진단]

```
ps -L -o  
pid,tid,stat,wchan:20  
(TID별 커널 대기 채널 추적)
```


아키텍처 보호 및 최소 권한의 원칙 (Isolation)



Docker 격리 메커니즘

Namespaces (프로세스 뷰 분리) & Cgroups (자원 -상한선 물리적 제한).

최소 권한의 원칙 (Least Privilege)

root 실행 원천 차단.
취약점 발생 시 호스트 장악을 막기 위해 agent-admin 일반 계정 강제.

12-Factor App 원칙

환경변수(Env) 주입을 통한 시나리오 제어 (하드코딩 배제).

트러블슈팅 커뮤니케이션: 5W1H와 GitHub Issue

When

Where

Who

What

How

Why

육하원칙 기반 객관적 사후 분석
(Post-Mortem)

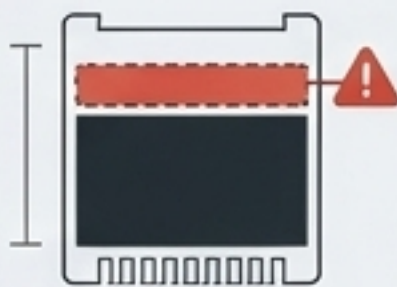
GitHub Issue 표준 4단계 구조

1. **Description**: 정확한 증상 및 발현 조건.
2. **Evidence & Logs**: 관제 수치, 터미널 캡처, 에러 로그 (객관적 증거).
3. **Root Cause Analysis**: OS 커널 메커니즘에 기반한 논리적 원인 규명.
4. **Workaround & Verification**: 환경변수 임시 조치 및 과학적 결과 검증.

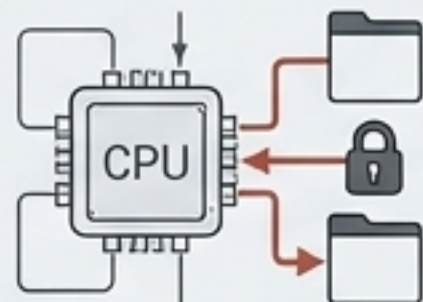
과학적 가설 검증: Before & After

Before (장애 발생)

MEMORY_LIMIT=100
(12초 생존, Exit 137) ⚠



MULTI_THREAD=true
(Hang 발생) ❄

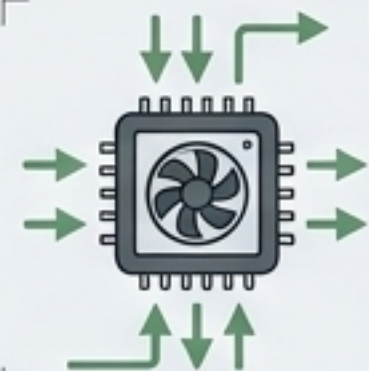


After (환경변수 조정 후)

MEMORY_LIMIT=512 ✓
(65초 생존, 5.4배 생존율 향상)

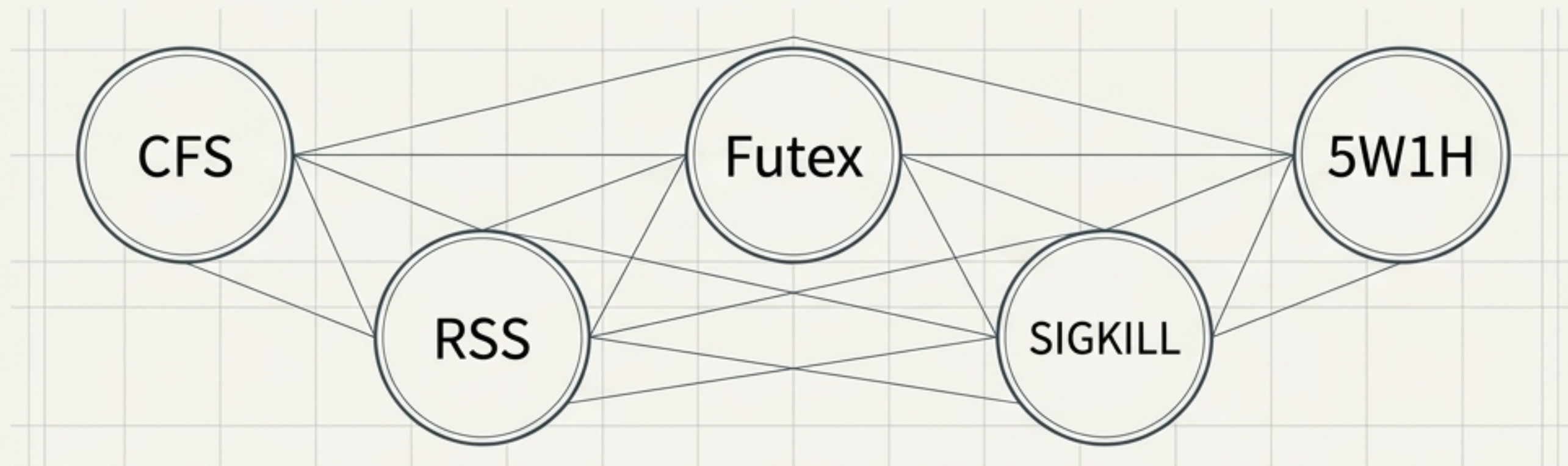


false 및
CPU_MAX_OCCUPY=20 조정 시
(정상 Cooldown 유지,
무중단 서빙 입증)



Insight: 엔지니어링은 직감이 아닌 수치 대조를 통해 가설의 적중을 과학적으로 증명하는 과정.

Wrap-up: 시스템 엔지니어의 통찰



관제 데이터라는 ‘객관적 증거’를 채증하고, OS의 ‘로우레벨 원리’로 원인을 추적하며, 템플릿화된 ‘이슈 리포트’로 소통하는 것이 고가용성 인프라 운영의 마스터키.

“재부팅은 해결책이 아니다. 데이터를 추적하라.”